

엔터핑 프로젝트 구조 안내

코드를 처음 보는 팀원을 위한 설명 문서

대상 독자: 코딩 경험이 있고, 이 프로젝트 코드는 처음 보는 팀원

원본 기술 문서: PROJECT_STRUCTURE.md (저장소 루트)

이 문서는 원본 기술 문서의 내용을 바탕으로, 처음 보는 사람도 이해할 수 있도록 설명을 추가해 재구성한 것입니다.

1. 이 프로젝트는 무엇인가

엔터핑은 일본 노래 가사로 타이핑 연습을 하고, 관련 퀴즈를 풀고, 다른 사용자와 실시간으로 대전할 수 있는 웹 서비스입니다. 백엔드는 Python의 FastAPI로 만들어져 있고, 프론트엔드는 별도 프레임워크 없이 HTML, CSS, JavaScript로 직접 작성되어 있습니다. 데이터는 두 종류의 데이터베이스에 나누어 저장합니다.

- **MySQL**: 회원 정보, 콘텐츠, 플레이 기록, 랭킹처럼 계속 남아있어야 하는 데이터를 저장합니다.
- **Redis**: 실시간 대전 방의 현재 상태, 이메일 인증번호처럼 일정 시간이 지나면 사라져도 되는 데이터를 저장합니다.

프로젝트 파일은 저장소 안의 `enter-typing/` 폴더에 들어 있습니다.

2. 요청이 처리되는 순서

사용자가 브라우저에서 페이지를 열거나 버튼을 눌렀을 때, 요청이 실제로 처리되는 순서는 다음과 같습니다.

1. 브라우저가 서버에 페이지(HTML) 또는 데이터(API)를 요청합니다.
2. `main.py` 에 등록된 라우터 중 하나가 그 요청을 받습니다. 라우터는 기능별로 `routers/` 폴더 안에 파일이 나뉘어 있습니다.
3. 라우터 코드가 필요하면 `models.py` 에 정의된 테이블을 통해 MySQL을 조회하거나, Redis에서 값을 읽고 씁니다.
4. 처리 결과를 HTML 페이지 또는 JSON 데이터로 브라우저에 돌려줍니다.
5. 브라우저는 받은 데이터를 `js/` 폴더의 JavaScript 코드로 화면에 반영합니다.

3. 백엔드 구조

3.1 최상위 파일

파일	설명
<code>main.py</code>	FastAPI 앱을 만들고, 모든 라우터를 연결하는 시작점입니다. 실제 기능 코드는 들어 있지 않습니다.
<code>database.py</code>	MySQL 연결 설정을 담당합니다. 연결 주소는 <code>.env</code> 파일의 <code>DATABASE_URL</code> 값을 사용합니다.
<code>models.py</code>	MySQL에 저장되는 모든 테이블의 구조를 정의합니다.
<code>requirements.txt</code>	이 프로젝트를 실행하는 데 필요한 Python 패키지 목록입니다.
<code>.env</code>	비밀번호, 인증 키 등 사람마다 다르게 설정해야 하는 값을 담는 파일입니다. 저장소에는 올라가지 않으며, 각자 로컬에서 직접 만들어야 합니다.

3.2 `core/` — 공통으로 쓰이는 기반 코드

파일	설명
<code>config.py</code>	로그인 토큰 설정, 이메일 발송 설정, Redis 주소 등 환경 설정 값을 모아둡니다.
<code>security.py</code>	로그인 토큰 발급/검증, 비밀번호 암호화·확인 기능을 담당합니다.
<code>redis_client.py</code>	Redis에 접속하는 두 가지 방식(동기/비동기)을 준비해 둡니다. 이메일 인증에는 동기 방식을, 실시간 대전에는 비동기 방식을 사용합니다.

3.3 `services/` — 특정 기능에 종속되지 않는 유틸리티 코드

파일	설명
<code>email_service.py</code>	이메일 발송, 인증번호 생성, 임시 비밀번호 생성 기능을 담당합니다.
<code>kana.py</code>	일본어 히라가나를 로마자로 바꾸는 변환 규칙을 담당합니다.

3.4 routers/ — 기능별 API

아래 파일 하나하나가 특정 기능을 담당하는 API 묶음입니다. 예를 들어 회원가입이나 로그인과 관련된 요청은 모두 `auth.py` 에서 처리합니다.

파일	담당 기능
<code>auth.py</code>	회원가입, 로그인, 이메일 인증, 비밀번호 찾기, 닉네임 변경, 회원 탈퇴, 출석 체크
<code>typing_content.py</code>	타이핑 콘텐츠 목록 조회, 생성, 수정, 삭제, 플레이 기록 저장
<code>quiz_content.py</code>	퀴즈 콘텐츠 목록 조회, 생성, 수정, 삭제, 플레이 기록 저장
<code>ranking.py</code>	종합 랭킹과 콘텐츠별 랭킹 조회. 같은 요청이 60초 안에 다시 들어오면 데이터베이스를 다시 조회하지 않고 이전 결과를 그대로 돌려줍니다.
<code>convert.py</code>	일본어 한자를 히라가나로, 히라가나를 로마자로 변환
<code>typo_stats.py</code>	오타 통계 조회 및 초기화
<code>profile.py</code>	마이페이지에 필요한 플레이 이력, 통계 데이터 제공
<code>battle.py</code>	실시간 대전 방 생성/조회, 실시간 통신(WebSocket) 전체를 담당
<code>pages.py</code>	각 HTML 페이지를 어떤 주소로 보여줄지 연결. 마이페이지(<code>/profile.html</code>)만 Jinja2라는 방식으로 여러 조각 파일을 조립해서 보여주고, 나머지 페이지는 파일을 그대로 전달합니다.

3.5 실시간 대전 데이터가 오가는 순서

1. 사용자가 방을 만들면, 제목과 인원수 같은 기본 정보만으로 Redis에 방 정보가 저장됩니다. 이 시점에는 아직 어떤 콘텐츠로 플레이할지 정해지지 않습니다.
2. 다른 사용자는 대전 로비 화면에서 방 목록을 보고 들어갈 방을 고릅니다.
3. 방에 들어가면 실시간 통신(WebSocket)이 연결되고, 현재 방 상태를 받습니다.
4. 방을 만든 사람만 타이핑 또는 퀴즈 콘텐츠를 선택할 수 있고, 선택하면 방에 있는 모든 사람에게 바로 전달됩니다.
5. 게임이 진행되는 동안 준비 완료, 시작, 진행 상황, 채팅 같은 정보가 실시간 통신으로

계속 오갑니다.

6. 게임이 끝나면 최종 결과가 MySQL에 기록으로 저장됩니다.

4. 프론트엔드 구조

프론트엔드 코드는 `html/`, `css/`, `js/` 세 폴더에 나뉘어 있습니다. 각 HTML 페이지는 화면에 보이는 마크업만 담고 있고, 실제 동작하는 로직은 같은 이름의 JavaScript 파일에 따로 정리되어 있습니다. 예를 들어 `typing_list.html` 이라는 화면의 동작은 `js/typing/list.js` 파일에 들어 있습니다.

4.1 주요 화면

화면	내용
회원 관련	로그인, 회원가입, 비밀번호 찾기/변경
타이핑	목록/검색, 상세 미리보기, 실제 플레이 화면, 콘텐츠 생성/수정 화면
퀴즈	목록/검색, 실제 플레이 화면, 콘텐츠 생성/수정 화면
실시간 대전	대전 로비(방 목록), 대기실부터 결과 화면까지 이어지는 대전 화면
랭킹	종합 랭킹, 곡별 랭킹
마이페이지	히스토리, 통계 분석, 내가 만든 콘텐츠 관리, 계정 설정
기타	홈 화면, 공지사항, 통합 검색

4.2 최근에 진행한 코드 정리 작업

아래 두 화면은 원래 하나의 JavaScript 파일에 2,000줄이 넘는 코드가 모두 들어 있었습니다. 읽고 수정하기 어려워서, 역할에 따라 여러 개의 작은 파일로 나누었습니다.

화면	나눈 방식
마이페이지	출석 관련 코드, 플레이 히스토리 코드, 통계/오타 분석 코드, 계정 설정 코드를 각각 별도 파일로 분리했습니다. 기존 파일은 이 파일들을 불러와서 초기 설정만 담당합니다.
실시간 대전	여러 화면이 함께 사용하는 상태 값, 실시간 통신 처리, 화면 갱신 처리, 게임 진행 로직을 각각 별도 파일로 분리했습니다. 기존 파일은 이 파일들을 불러와서 초기 설

또한 여러 화면에서 유튜브 영상을 재생할 때 각자 다른 방식으로 유튜브 재생 기능을 불러오고 있었는데, 이 부분을 하나의 공용 파일(`js/shared/youtube_manager.js`)로 합쳤습니다. 여러 화면이 동시에 유튜브 재생 기능을 준비할 때 초기화 순서가 겹치면서 오류가 발생하는 문제를 막기 위한 작업입니다.

4.3 마이페이지 화면 조립 방식

마이페이지(`profile.html`)는 화면 안에 탭이 다섯 개, 팝업창(모달)이 두 개 있는 큰 화면입니다. 한 파일 안에 모든 내용이 들어 있으면 원하는 부분을 찾기 어려워서, 탭과 팝업창 내용을 각각 별도 파일로 나누어 `html/partials/` 폴더에 두었습니다. 서버가 마이페이지를 보여줄 때, 이 조각 파일들을 정해진 위치에 자동으로 끼워 넣어서 하나의 완성된 화면으로 만듭니다. 헤더, 하단 영역, 탭 버튼처럼 화면 전체에 공통으로 쓰이는 부분은 원래 파일에 그대로 남아 있습니다.

5. 알려진 문제

아직 해결되지 않은 구조적인 문제를 정리했습니다. 화면 단위의 자잘한 버그와 앞으로 추가하면 좋을 기능 목록은 저장소의 `remind.txt` 파일에 별도로 정리되어 있습니다.

- **콘텐츠 수정/삭제 권한 검사 미비.** 현재 타이핑 퀴즈 콘텐츠를 수정하거나 삭제하는 기능에 작성자 확인 절차가 빠져 있습니다. 콘텐츠 번호만 알면 다른 사람이 만든 콘텐츠도 수정·삭제할 수 있는 상태이므로, 배포 전에 반드시 권한 검사를 추가해야 합니다.
- **오디오 기반 퀴즈 화면 동작 불가.** `quiz_detail.html` 화면은 현재 존재하지 않는 API를 호출하고 있어 정상적으로 동작하지 않습니다. 관련 API를 새로 만들거나, 이 화면을 없애는 방향을 결정해야 합니다.
- **퀴즈 기록에 콘텐츠 번호 누락.** 퀴즈 플레이 기록을 저장할 때 어떤 콘텐츠를 플레이했는지 번호가 함께 전달되지 않아, 콘텐츠별 최고 점수가 갱신되지 않습니다.

6. 최근 작업 요약

- 랭킹 조회 속도 개선: 60초 동안 같은 결과를 재사용하는 캐시 추가, 반복 플레이로 점수가 끝없이 쌓이던 계산 방식을 콘텐츠별 최고 점수 합산 방식으로 변경
- 여러 화면에서 각자 다르게 처리되던 유튜브 영상 재생 초기화 로직을 하나의 공용 파일로 통합
- 마이페이지, 실시간 대전 화면의 대형 JavaScript 파일을 역할별로 여러 개의 작은 파일로 분리

- 마이페이지 HTML 마크업을 탭/팝업창 단위로 분리해, 서버에서 조립해 보여주는 방식으로 변경
- 존재하지 않는 이미지 파일을 기본 이미지로 사용하려다 반복적으로 실패하던 문제 수정
- 백엔드 코드(`main.py`) 전체를 기능별 파일로 분리 (2,200줄이 넘는 파일 한 개 → 라우터 파일 9개)
- 모든 HTML 페이지에서 화면 안에 직접 작성되어 있던 JavaScript 코드를 기능별 외부 파일로 분리
- 타이핑/퀴즈/실시간 대전 페이지 주소를 짧고 알아보기 쉬운 형태로 정리 (예: `/typing/{콘텐츠 번호}/play`)